

PORTFOLIO PDF

이지은

Senior iOS Application Developer

커머스·콘텐츠·플랫폼 도메인에서 10년 이상 모바일 앱을 개발하고 운영했습니다. 운영 안정화, 레거시 개선, SwiftUI 전환, 디자인 시스템 구축, AI-assisted workflow를 실무 맥락에 맞게 활용합니다.

amy.lee.d2@gmail.com[+82 10-6345-4342](tel:+821063454342)

Senior Summary

DOMAIN

커머스·콘텐츠·플랫폼 도메인에서 총 10년 이상의 모바일 앱 개발 경험을 쌓았습니다.

MODERNIZATION

운영 중인 iOS 앱의 구조 개선을 주도했으며, Objective-C 레거시 코드 분석부터 SwiftUI 전환까지 단계적으로 수행했습니다.

ARCHITECTURE

Objective-C, UIKit, SwiftUI, RxSwift, Combine을 실무에서 사용했으며, Clean Architecture / MVI / MVVM 기반 설계 경험이 있습니다.

WORKFLOW

ONestore에서 iOS 디자인 시스템과 제3자 마켓 Prototype의 iOS 개발 범위를 조율하며, AI 도구를 코드 분석·설계 검토·테스트 작성에 활용했습니다.

RECENT

최근에는 iOS 포트폴리오 재정비와 개인 프로젝트를 진행하며 SwiftUI, VisionKit, OpenAI API 기반 앱 구조 설계와 App Store 배포까지 이어갔습니다.

Core Strengths

01 ARCHITECTURE

iOS App Architecture

Clean Architecture, MVVM, Coordinator, DI 기반 설계 경험. 화면 단위 구현이 아닌 유지보수 가능한 구조를 설계합니다.

02 OPERATION

App Operation & Stability

정기·긴급 배포, Crashlytics, API 로깅, OS 업데이트, deprecated API 관리 등 운영 중인 앱의 안정성을 지키는 경험이 있습니다.

03 REFACTORING

Legacy Modernization

UIKit 기반 레거시 코드 분석 및 구조 개선. 리팩토링 범위 판단과 단계적 전환 설계를 수행했습니다.

04 MIGRATION

SwiftUI Migration

UIKit → SwiftUI 전환 설계 및 구현. MVVM / MVI 기반으로 신규 화면을 구축하며 전환 방향을 정의했습니다.

05 PLATFORM

Design System & Platformization

공통 UI 컴포넌트, 디자인 토큰, Figma Code Connect를 활용해 디자인-개발 협업 기준을 정리했습니다.

06 AI WORKFLOW

AI-assisted Development Workflow

AI 도구를 레거시 분석, 설계 검토, 테스트 케이스 작성에 활용하되 설계 판단과 품질 검증은 직접 수행합니다.

Career Details

디지털다임

digitalDigm

2014.12 ~ 2017.05

2년 6개월

사원

Role Summary

Android 기반 SI/SM 프로젝트를 수행하며 모바일 앱 개발의 기본기와 다양한 직군 협업 경험을 쌓았습니다.

웹에이전시 환경에서 기획, 디자인, 서버/웹 개발자와 요구사항을 맞추며 커머스, 브랜드, O2O, 위치 기반 서비스 앱을 개발했습니다.

Main Responsibilities

- Android 앱 화면 구현, API 연동, WebView/JavaScript Interface 연동을 담당했습니다.
- 외부 SDK, 소셜 로그인, 결제, 위치 기반 기능을 프로젝트 요구사항에 맞게 연동했습니다.
- SI/SM 프로젝트에서 기획서와 디자인을 기준으로 구현 범위와 예외 케이스를 맞춰가며 개발했습니다.

Key Projects

- CafeUnion** : 중국 시장 대상 O2O 커머스 앱. 위치 기반 매장 탐색, 소셜 로그인, 결제 SDK 연동 경험.
- LuxeWater** : 중국 마켓 대상 브랜드/하이브리드 앱. WebView, JavaScript Interface, 현지 SDK 연동 경험.
- 올가홀푸드** : 식품/커머스 브랜드 앱. 상품 탐색, 바코드/QR 인식, 푸시 SDK 연동 경험.
- MyD2** : 사내 커뮤니케이션 앱. 업무용 앱 화면 구현, API 연동, 푸시 기능 연동 경험.

Technical Focus

- Android Java, WebView, 외부 SDK, 위치 기반 기능, 푸시, 소셜 로그인/결제 연동
- 여러 도메인의 요구사항을 모바일 앱 구조로 구현하는 경험

What This Experience Shows

초기 Android/SI 경험은 이후 iOS 개발자로서 플랫폼 차이, 운영 구조, 협업 흐름을 이해하는 기반이 되었습니다.

Android

Java

Eclipse

Git

SVN

Trello

GS ITM



2017.06 ~ 2021.09

4년 4개월

대리

Role Summary

GS Retail 모바일 앱 운영 조직에서 iOS/Android 앱 운영과 개발을 담당했습니다.

정기/긴급 배포, OS 업데이트 대응, VOC/Crash 대응을 수행했고, 별도 GS Fresh 차세대 프로젝트에서는 모바일 파트 PL로 Android/iOS 개발 범위와 구현 방향 정리를 맡았습니다.

Main Responsibilities

- GS Retail 커머스/멤버십 앱 운영과 GS 계열 모바일 SI 프로젝트를 수행했습니다.
- GS Retail 커머스 및 멤버십 앱의 정기 배포와 긴급 배포를 대응했습니다.
- VOC, Crash Report, 신규 OS 업데이트, Deprecated API 대응을 수행했습니다.
- 운영 앱 개선 요청과 성능/안정성 이슈를 확인하고 수정했습니다.
- GS Fresh 차세대 프로젝트에서 모바일 파트 PL로 Android/iOS 개발 범위, 구현 문서, 협업 커뮤니케이션을 정리했습니다.

Key Projects

- **GS Retail Mobile App Operation** : GS Fresh Mall, 달리살다, 마켓포, THE POP 등 커머스/멤버십 앱 운영.
- **GS Fresh 차세대 프로젝트** : 모바일 파트 PL로 SSO, One-Source Multi Application, SDK/API/WKWebView 연동 범위 담당.

Technical Focus

- iOS/Android 운영, 배포 대응, Crash 분석, OS 업데이트 대응
- One-Source Multi Application, Scheme / Environment 분리, SSO, 외부 SDK 연동

What This Experience Shows

운영 중인 모바일 서비스의 배포·장애 대응 기준을 익히고, Android/iOS 양쪽의 구현 범위를 조율한 경험을 보여줍니다.

iOS

Android

Swift

Objective-C

RxSwift

Java

Kotlin

Firebase

Crashlytics

Jira

Confluence

원스토어

ONEstore

2021.09 ~ 2023.12

2년 4개월

팀원

Role Summary

ONEstory iOS 앱 운영과 기능 개발을 담당하며 콘텐츠 앱의 운영 안정성과 사용자 기능 개선을 경험했습니다.

UIKit 기반 화면 개선, WebView 연동, iPad 대응, 위젯 등 콘텐츠 앱 운영에 필요한 기능을 개발했습니다.

Main Responsibilities

- ONEstory iOS 앱의 정기 배포, VOC 대응, OS 업데이트 대응을 수행했습니다.
- WebView로 구성되어 있던 화면을 UIKit Native 전환, 내서재/마이페이지/고객문의 등 주요 화면 개발을 담당했습니다.
- 소셜 로그인 SDK, JavaScript 연동, 위젯, iPad UI 대응 등 콘텐츠 앱 기능을 개발했습니다.

Key Projects

- **ONEstory iOS App Operation** : 콘텐츠 앱 운영, 화면 개선, SDK 연동, 위젯 및 iPad 대응.
- **WebView to Native** : 구매목록 등 WebView 기반 화면의 Native 전환 경험.

Technical Focus

- UIKit, Swift, Objective-C, SwiftUI, WebView/JavaScript 연동
- 정기 운영, VOC 대응, OS 업데이트 및 Deprecated API 대응

What This Experience Shows

콘텐츠 도메인 iOS 앱을 운영하며 레거시 화면 개선과 기능 확장을 병행한 경험을 보여줍니다.

iOS

Swift

UIKit

Objective-C

SwiftUI

Bitbucket

Confluence

Jira

Teams

Zeplin

원스토어

1 ONEstore

2024.01 ~ 2025.06

1년 6개월

팀원

Role Summary

ONEstore iOS Platform팀에서 글로벌 앱 마켓 **Prototype**의 iOS 구현 범위와 디자인 시스템의 iOS 개발 파트 조율 역할을 수행했습니다.

Apple DMA 관련 기술 검토, Prototype/Alpha/Beta 개발 흐름, 공통 UI 컴포넌트와 디자인 토큰 기반 협업 기준을 iOS 개발 범위 안에서 정리했습니다.

Main Responsibilities

- ONEstore Global App Market Prototype에서 정책/기술 검토, 구현 리스크 정리, 단계별 iOS 개발 범위 조율을 맡았습니다.
- Clean Architecture/MVVM 기반 앱 구조, 네트워크 모듈, JavaScript Interface 구조를 설계했습니다.
- ONEstore Design System에서 iOS 개발 파트 기준으로 컴포넌트 구조, 디자인 토큰, Figma Code Connect 흐름을 정리하고 협업 커뮤니케이션을 지원했습니다.

Key Projects

- ONEstore Design System** : 공통 UI 컴포넌트, 디자인 토큰, Figma Code Connect 기반 디자인-개발 협업 구조.
- ONEstore Global App Market Prototype** : Apple DMA 관련 기술 검토, MarketplaceKit 제약 확인, Prototype → Alpha/Beta 개발.

Technical Focus

- SwiftUI, Clean Architecture, MVVM, Tuist, MarketplaceKit 검토
- Design Token, Style Dictionary, Figma Code Connect, 공통 UI 컴포넌트 구조

What This Experience Shows

신규 플랫폼 대응 과정에서 iOS 구현 리스크를 정리하고, 디자인 시스템 기반 플랫폼화 작업을 개발 범위 안에서 조율한 경험을 보여줍니다.

iOS

Swift

SwiftUI

Clean Architecture

MVVM

Tuist

Figma

Design Token

Code Connect

Cursor

Copilot

엔카닷컴



2025.06 ~ 2025.08

3개월

과장

Role Summary

엔카 iOS 앱 운영 개발 과정에서 UIKit 레거시 화면의 SwiftUI 전환과 운영 안정성 개선을 담당했습니다.

제네시스 옵션 필터 화면 전환, 광고 모듈 통합, Crashlytics/API 로깅 개선처럼 운영 중인 앱에서 많은 개선 범위와 산출물을 중심으로 작업했습니다.

Main Responsibilities

- 기존의 UIKit으로 구현된 제네시스 옵션 필터 화면을 SwiftUI + MVVM 구조로 전환했습니다.
- AppsFlyer, Braze 등 광고 모듈 연동 코드를 공통 구조로 정리했습니다.
- Crashlytics 로깅 중앙화와 API 요청/응답 로깅 개선을 수행했습니다.

Key Projects

- **Encar iOS App** : UIKit 레거시 화면 개선, SwiftUI 전환, 광고 모듈 통합, 운영 로깅 개선.

Technical Focus

- SwiftUI, UIKit, MVVM, Unit Test, SnapKit, Figma, Cursor
- Crashlytics, AppsFlyer, Braze, API logging, 운영 안정성 개선

What This Experience Shows

운영 중인 iOS 앱에서 레거시 구조를 파악하고, 필요한 범위를 정해 점진적으로 개선하는 역량을 보여줍니다.

iOS

Objective-C

Swift

SwiftUI

UIKit

MVVM

Crashlytics

AppsFlyer

Braze

Alamofire

SnapKit

Cursor

Copilot

Featured Projects

웹 포트폴리오의 대표 프로젝트 모달 내용을 **PDF** 제출용으로 펼친 섹션입니다.

ONEstore Design System

2024.06 ~ 2025.06

ONEstore 디자인 시스템 구축 프로젝트에서 iOS 개발 파트 기준으로 공통 UI 컴포넌트 구조, 디자인 토큰, Figma 연계 흐름을 정리하고 디자인-개발 협업 기준을 맞추는 작업을 진행했습니다.

iOS

Swift

SwiftUI

Tuist

Figma

Design Token

Style Dictionary

Code Connect

Lottie

Overview

- ONEstore 제품군에서 공통 UI 컴포넌트와 디자인 리소스를 일관되게 사용하기 위한 디자인 시스템 구축 프로젝트입니다.
- 단순 UI 공통화가 아니라, 디자인 기준과 iOS 구현 구조가 함께 맞물리도록 협업 흐름을 정리하는 데 초점을 맞췄습니다.

Problem

- 디자이너와 개발자가 컴포넌트 단위와 속성을 다르게 해석해 구현 기준을 맞추기 어려웠습니다.
- 디자인 토큰과 리소스 변경이 수작업 전달에 의존하면 반복 비용과 누락 위험이 커질 수 있었습니다.

My Role

Role Scope

iOS 개발 파트 기준으로 UXD팀과 개발 조직 사이에서 컴포넌트 구조, 구현 기준, Figma 연계 흐름을 맞추는 조율 역할을 수행했습니다.

- 컴포넌트 구조, 구현 기준, Style-Component-Screen 프레임워크 구조를 iOS 관점에서 정리했습니다.
- Figma Plugin, Style Dictionary, Code Connect 기반의 디자인 자산-코드 연결 방식을 검토했습니다.

Key Contributions

- ONEstore 제품과 협업 방식에 맞는 Style-Component-Screen 구조를 정리했습니다.
- Top Navigation 등 실제 요소를 기준으로 컴포넌트 단위, Item, Property 기준을 구현 규칙으로 연결했습니다.
- WDS 프레임워크 구조와 디자인 토큰/Code Connect 연계 흐름을 검토했습니다.

Technical Decisions

- 디자인 시스템을 Style, Component, Screen 계층으로 나누어 토큰, 컴포넌트, 화면 조합의 책임을 분리하는 방향을 선택했습니다.
- 컴포넌트 정의는 디자인 명세만 맞추는 것이 아니라 실제 SwiftUI 구현에 필요한 Property, Item, 상태 조합 기준까지 함께 정리했습니다.
- 디자인 토큰과 Code Connect는 디자이너와 개발자가 같은 컴포넌트 기준을 확인할 수 있도록 Figma 자산과 iOS 구현 예시를 연결하는 방향으로 검토했습니다.

Visual Evidence

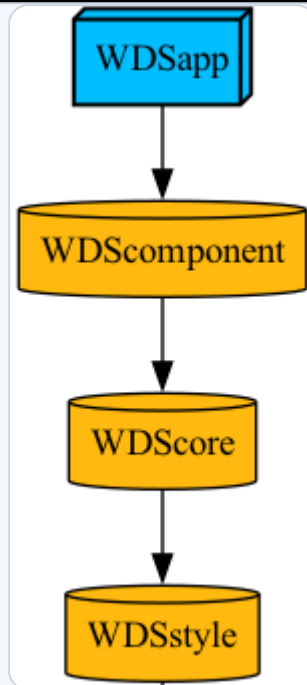
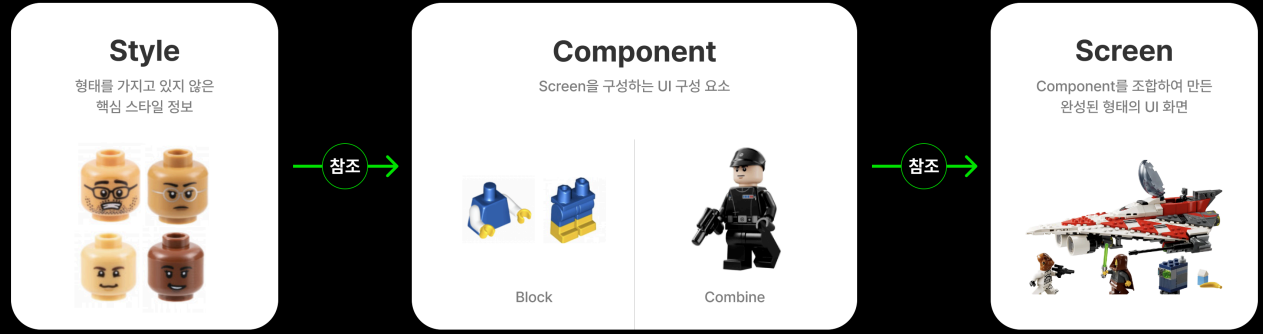
- 구조, 컴포넌트, 토큰 자동화, Code Connect 흐름을 설명하는 핵심 자료입니다.

DESIGN SYSTEM ARCHITECTURE

WDS Architecture

작은 단위에서부터 확장되는 논리적인 컴포넌트 구조, 익숙하고 쉬운 명칭 지정

스타일과 컴포넌트를 중앙에서 일괄적으로 관리하기 때문에 유지보수가 수월하며 일관된 커뮤니케이션이 가능



COMPONENT DEFINITION EXAMPLE

_item / Button-Left

이름: TopNavItemLeft, State: Pressed, State: Disabled

_item / Button-Right

이름: TopNavItemRight, State: Pressed, State: Disabled

이름: TopNavOptionPopup, State: Pressed, State: Disabled

```

TopNavigation
├── Item
│   ├── TopNavItemLeft
│   ├── TopNavItemRight
│   └── TopNavOptionPopup
└── Modifier
    ├── EditTopNavigation
    └── MyTopNavigation

TopNavigation
├── Item
│   ├── TopNavItemLeft
│   ├── TopNavItemRight
│   └── TopNavOptionPopup
└── Modifier
    ├── EditTopNavigation
    ├── MyTopNavigation
    └── TopNavigation

public struct TopNavItemLeft: View {
    public init(imgResourceName: String, action: @escaping () -> Void) {
        self.imgResourceName = imgResourceName
        self.action = action
    }
}

public enum TopNavItemType {
    case Icon, Text, Thumbnail
}

public struct TopNavItemRight: View {
    let type: TopNavItemType

    var imgResourceName: String? = nil
    @Binding var isShowBadge: Bool

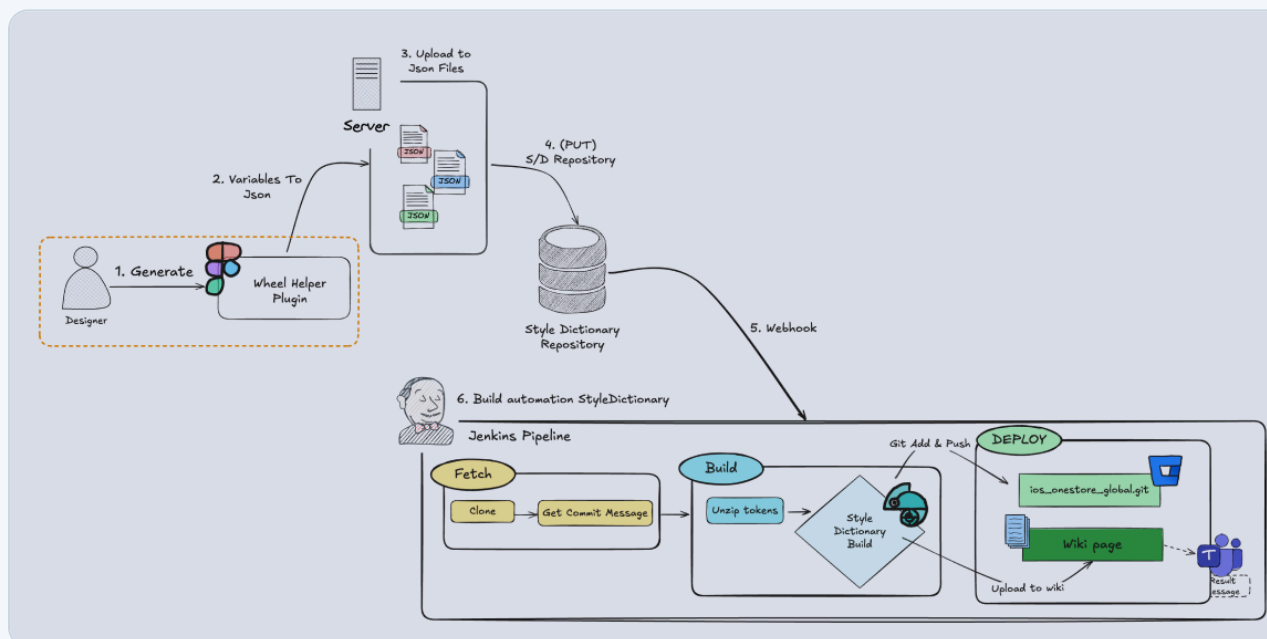
    var strText: String? = nil

    var thumbnailImgUrl: String? = nil
    var isAdult: Bool? = nil

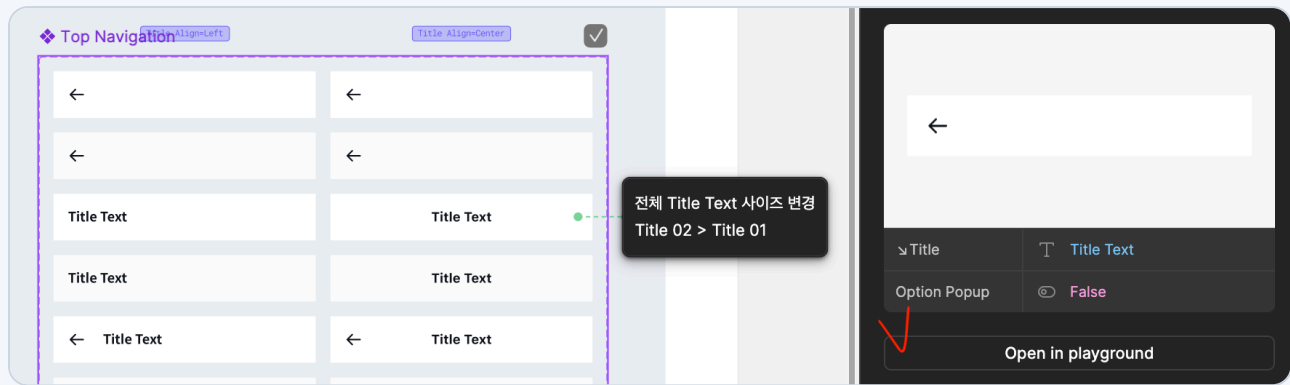
    var action: () -> Void

    public init(imgResourceName: String, isShowBadge: Binding<Bool> =
        .constant(false), action: @escaping () -> Void) {
        self.type = .Icon
        self.imgResourceName = imgResourceName
    }
        
```

DESIGN TOKEN / RESOURCE AUTOMATION



FIGMA CODE CONNECT



Result / Impact

- 공통 컴포넌트, 디자인 토큰, Code Connect 기준을 통해 디자이너와 개발자가 같은 단위로 논의할 수 있는 기준을 정리했습니다.
- 디자인 시스템을 단순 UI 공통화가 아니라 iOS 구현 구조와 협업 커뮤니케이션을 연결하는 플랫폼화 작업으로 확장했습니다.

ONestore Global App Market Prototype 개발

2024.01 ~ 2024.12

ONestore 글로벌 앱 마켓 프로젝트에서 Apple DMA 관련 제3자 마켓 기술 검토를 진행하고, Prototype부터 Alpha, Beta까지 iOS 구현 범위와 리스크를 정리했습니다.

iOS

Swift

SwiftUI

Clean Architecture

MVVM

Tuist

MarketplaceKit

JavaScript Interface

Figma

CryptoSwift

Overview

- Apple DMA 정책 변화에 대응해 ONestore 글로벌 대체 앱 마켓의 iOS 구현 가능성을 Prototype, Alpha, Beta 단계로 검증한 프로젝트입니다.

Problem

- Apple의 제3자 마켓 정책, MarketplaceKit 제약, Hosted App 다운로드/설치/업데이트 흐름의 구현 리스크를 정리해야 했습니다.

My Role

Role Scope

정책/기술 검토, 앱 아키텍처 설계, Prototype에서 Alpha/Beta로 이어지는 iOS 구현 범위와 리스크 정리를 담당했습니다.

- 제3자 마켓 기술 검토, 앱 아키텍처, 네트워크 모듈, JavaScript Interface 구조를 담당했습니다.
- Prototype/Alpha/Beta 단계별 구현 범위와 리스크를 정리하고, Apple Cork 기술 세션에서 주요 이슈를 구현 관점으로 확인했습니다.

Key Contributions

- Apple DMA 문서를 검토해 서비스 적용 조건과 리스크를 정리했습니다.
- Clean Architecture/MVVM 기반 앱 구조, 네트워크 모듈, JavaScript Interface 공통 기반을 구성했습니다.
- Hosted App 목록, 다운로드 상태, 설치/업데이트/실행/삭제 시나리오를 단계적으로 확장했습니다.

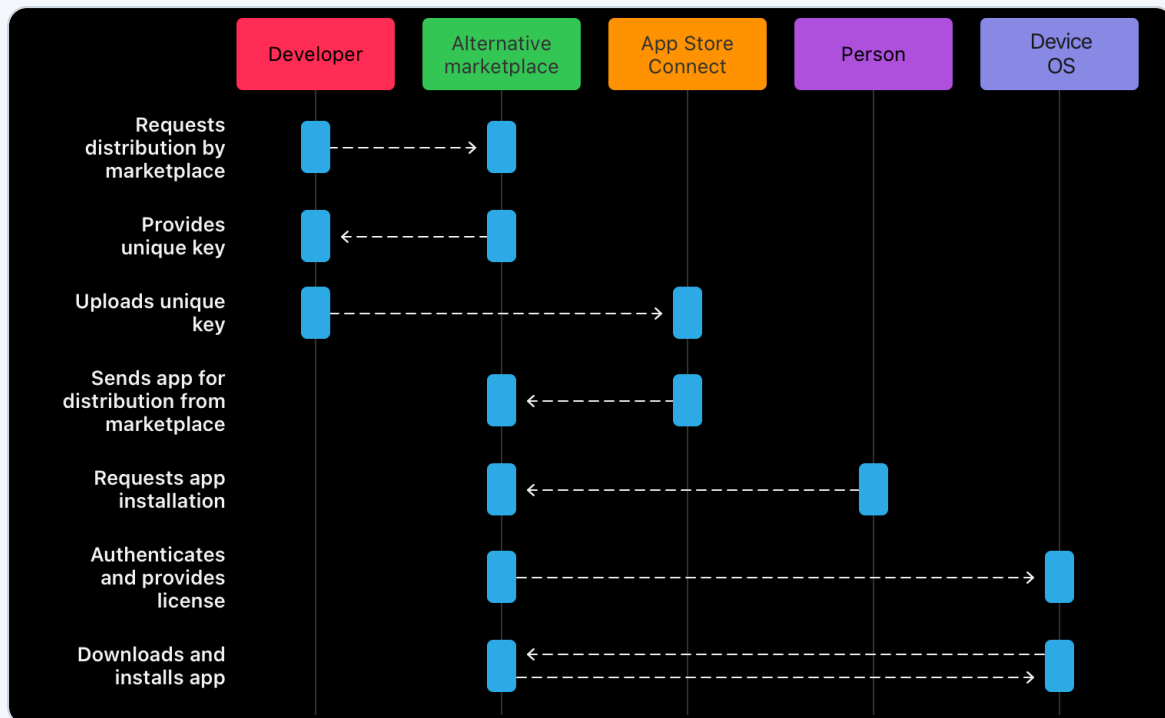
Technical Decisions

- 구현 전에 Apple 정책 제약, MarketplaceKit/API 제공 범위, 서비스 요구사항의 충돌 지점을 먼저 분리했습니다.
- 마켓 메타데이터, 앱 상태, 웹-앱 이벤트 연동 경계를 나누어 정책 검토 영역과 앱 구현 영역이 섞이지 않도록 설계했습니다.

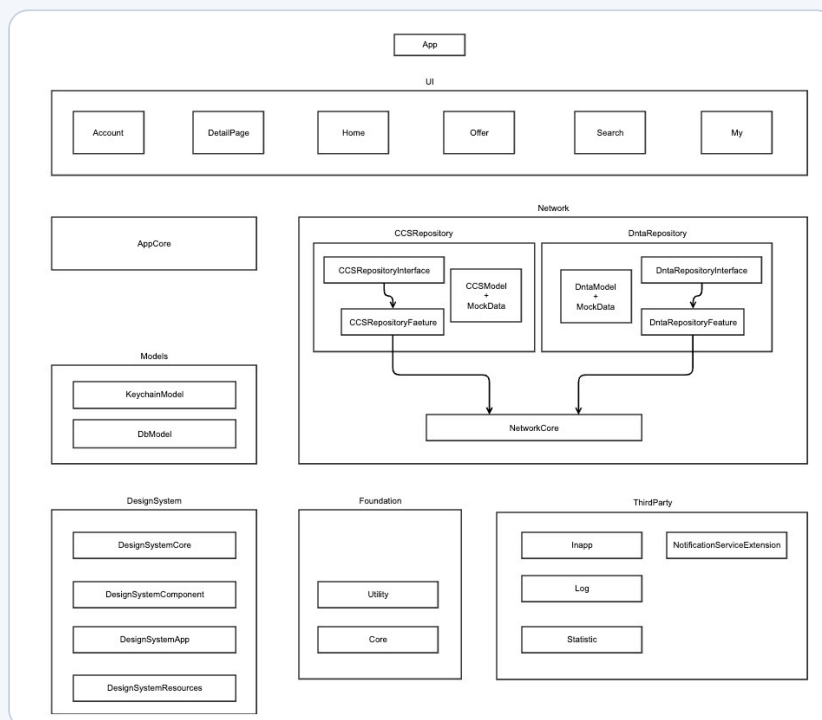
Visual Evidence

- 정책 검토, 앱 구조, 코드 예시, 결과 화면 자료입니다.

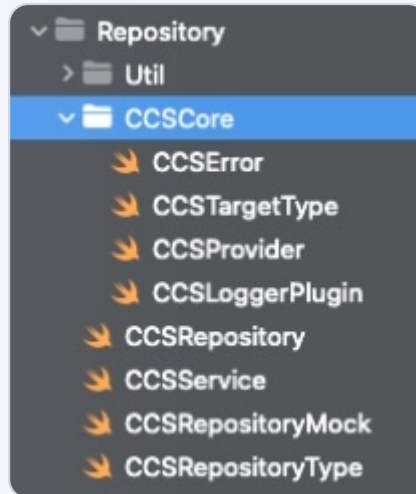
APPLE ALTERNATIVE MARKETPLACE 검토 자료



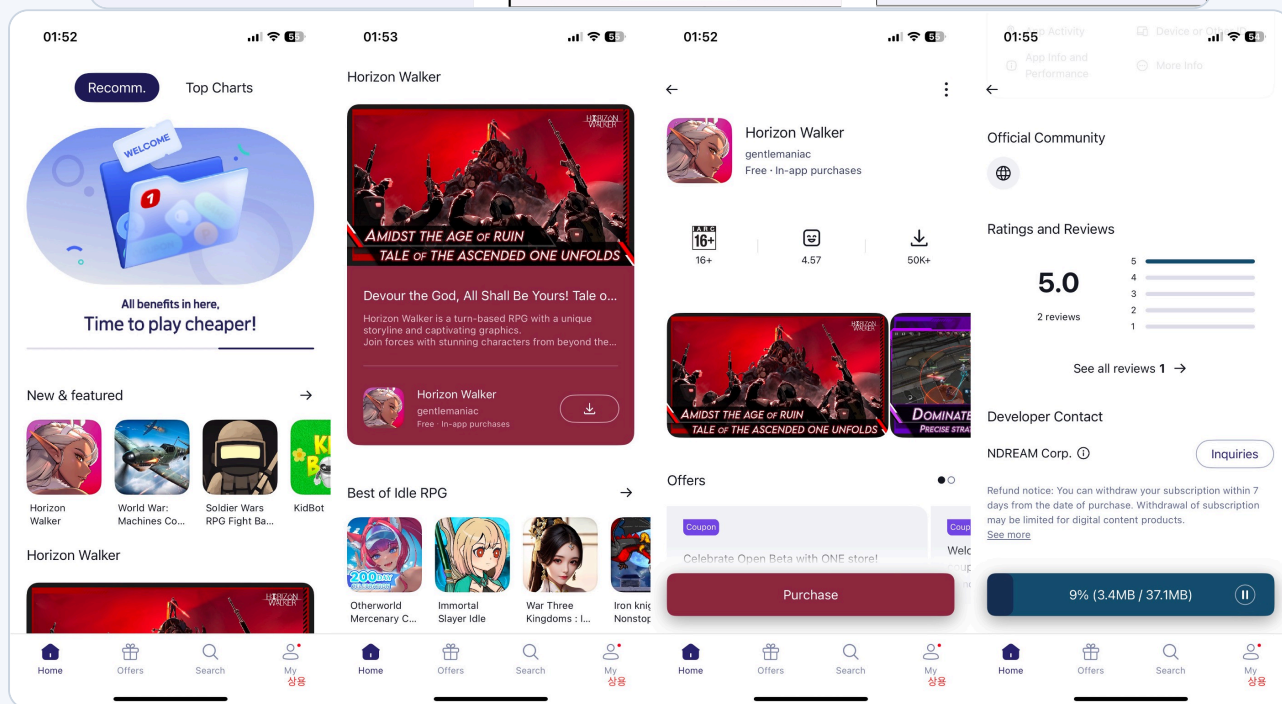
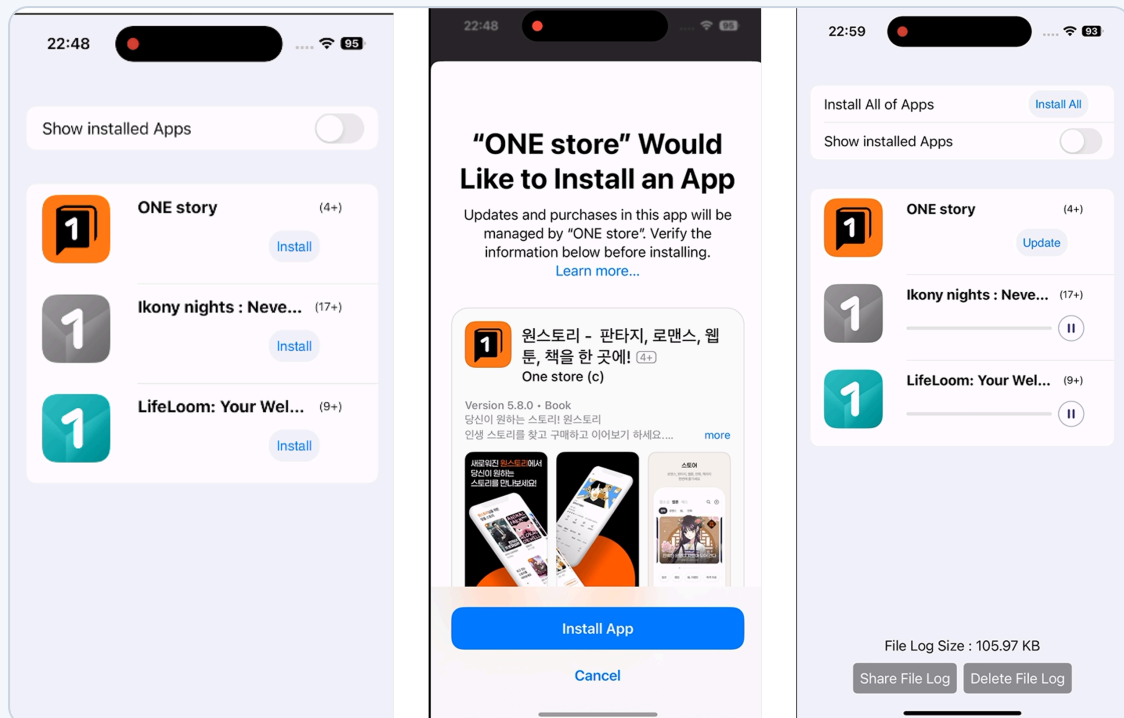
ARCHITECTURE / MODULE STRUCTURE



NETWORK MODULE CODE EXAMPLE



PROTOTYPE / RESULT SCREENS



Result / Impact

- Apple DMA 대응 가능성과 MarketplaceKit 기반 iOS 구현 리스크를 Prototype → Alpha/Beta 단계에서 앱 흐름으로 검토했습니다.
- 정책 제약, 네트워크 모듈, 웹-앱 이벤트 경계를 분리해 후속 단계에서 확인할 기술 리스크를 정리했습니다.

엔카닷컴 iOS 앱 개발 및 운영

2025.06 ~ 2025.08

운영 중인 엔카 iOS 앱에서 UIKit 레거시 화면을 SwiftUI + MVVM 구조로 전환하고, 광고 모듈과 로깅 구조를 정리해 운영 안정성을 높이는 개선 작업을 수행했습니다.

iOS

Objective-C

Swift

SwiftUI

UIKit

MVVM

AppsFlyer

Braze

Firebase

Crashlytics

Alamofire

SnapKit

Cursor

Copilot

Overview

- 엔카 iOS 앱의 운영 개발 과정에서 레거시 UIKit 화면 개선과 SwiftUI 전환, 운영 안정성 개선을 함께 수행한 프로젝트입니다.
- 기존 앱 구조를 파악한 뒤, 맡은 화면 전환 범위와 운영 코드 정리 범위를 분리해 진행해야 했습니다.

Problem

- 차량 옵션 필터 화면은 UIKit 기반 레거시 구조로 유지되고 있어 신규 UI 요구사항과 테스트 구조를 반영하기 어려웠습니다.
- AppsFlyer, Braze 등 광고 모듈 연동 코드가 여러 파일에 분산되어 있어 변경 영향 범위 파악과 유지보수가 비효율적이었습니다.
- 운영 중 장애 원인을 빠르게 추적하기 위해 Crashlytics 로깅과 API 요청/응답 로그 구조를 정리할 필요가 있었습니다.

My Role

- 제네시스 옵션 필터 화면의 SwiftUI + MVVM 전환 범위를 담당했습니다.
- 광고 모듈 통합 구조, Crashlytics 로깅 중앙화, API 로깅 개선처럼 운영 추적에 필요한 산출물을 정리했습니다.
- 정기 배포와 운영 이슈 대응 과정에서 맡은 변경 범위의 안정성 확인을 함께 진행했습니다.

Key Contributions

- UIKit 기반 화면을 SwiftUI + MVVM 구조로 재구성하고, 주요 유스케이스 기준의 Unit Test를 작성했습니다.
- AppsFlyer, Braze 연동 코드를 단일 구조로 정리해 중복 코드를 줄이고 모듈 추가/교체가 쉬운 형태로 개선했습니다.
- Crashlytics 크래시 로깅 로직을 중앙화하고, API 요청/응답 로그를 추가해 운영 중 디버깅 흐름을 개선했습니다.

Technical Decisions

- 신규 화면은 SwiftUI를 적용하되, 기존 UIKit 앱과의 연결 지점을 고려해 MVVM 기반으로 상태와 화면 로직을 분리했습니다.
- 광고/분석 SDK 연동은 화면별 분산 호출보다 공통 진입점에서 관리하는 편이 변경 영향 범위를 추적하기 쉽다고 판단했습니다.
- Crashlytics와 API 로깅은 단순 로그 추가가 아니라 재현 조건, 요청/응답 흐름, 사용자 영향 범위를 파악하기 쉬운 지점을 기준으로 수집 위치를 조정했습니다.

Screenshots / Result

- UIKit 레거시 화면을 SwiftUI 기반 화면으로 전환한 결과를 면접 중 시각적으로 설명할 수 있는 자료입니다.

SWIFTUI 전환 결과 화면

Result / Impact

- UIKit 레거시 화면을 SwiftUI + MVVM 구조로 전환한 산출물을 남겨, 유사 화면 전환 시 참고할 수 있는 구현 기준을 만들었습니다.
- 광고 모듈 진입점과 Crashlytics/API 로깅 위치를 정리해 운영 이슈의 재현 조건과 영향 범위를 파악하기 쉬운 구조로 개선했습니다.

GS Retail Mobile App Operation & GS Fresh Renewal

2018.08 ~ 2021.09

GS ITM에서 GS Retail 커머스/멤버십 모바일 앱 운영을 담당했고, 별도 GS Fresh 차세대 커머스 프로젝트에서는 모바일 파트 PL로 Android/iOS 개발 범위와 구현 방향을 정리했습니다.

iOS

Android

Swift

Objective-C

RxSwift

Firebase

Crashlytics

Alamofire

Moya

Jira

Confluence

Overview

- GS Retail 커머스/멤버십 앱 운영과 GS Fresh 차세대 프로젝트의 모바일 범위 정리 경험입니다.

Track 1

GS Retail Mobile App Operation

Operation

Problem

- 여러 커머스/멤버십 앱의 배포, 장애, OS 변경 대응을 안정적으로 처리해야 했습니다.

My Role

- GS Retail 앱 운영 조직에서 iOS/Android 배포, VOC, OS 업데이트, Crash 분석을 수행했습니다.

Key Contributions

- GS Fresh Mall, 달리살다, 마켓포, THE POP 등 커머스/멤버십 앱의 정기/긴급 배포와 VOC 기반 버그 수정을 담당했습니다.
- 신규 OS와 Deprecated API 대응, Crash 분석을 운영 흐름 안에서 처리했습니다.

Technical Decisions

- 정기/긴급 배포, VOC, Crash 대응은 사용자 영향도와 배포 리스크를 기준으로 우선순위를 나누었습니다.
- 신규 OS와 Deprecated API 대응은 앱별 영향 범위를 먼저 확인한 뒤 배포 일정과 함께 조정했습니다.

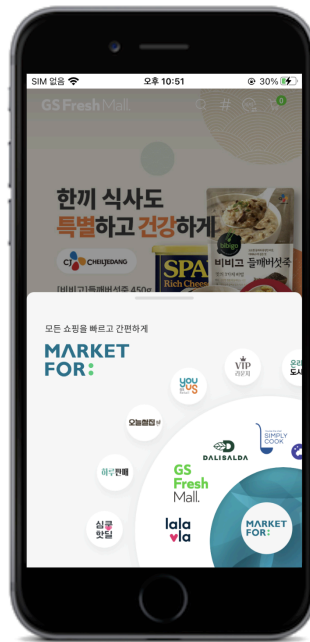
Screenshots / Result

- 운영 대상 앱의 대표 화면입니다.

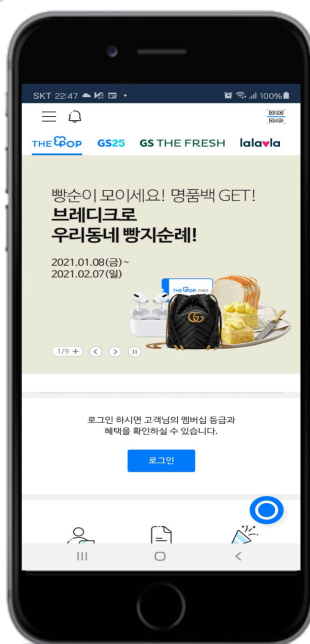
GS RETAIL 운영 앱 화면



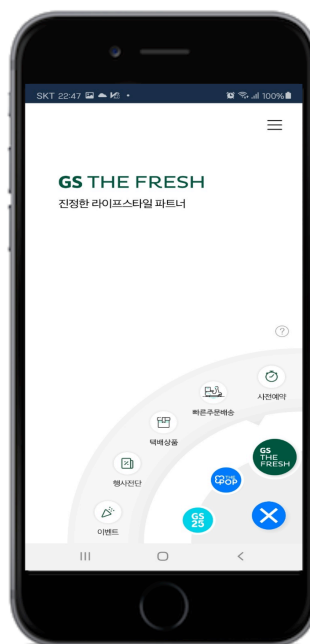
[GS프레시몰]



[달리살다]



[THE POP]



[GS THE FRESH]



[나만의냉장고]

Result / Impact

- 정기/긴급 배포, VOC, Crash, OS 대응을 반복하며 운영 중인 모바일 서비스의 안정성을 유지하는 기준을 쌓았습니다.

Track 2

GS Fresh Next-generation Commerce App

PL · SI

Problem

- 오픈 전 모바일 산출물, SSO, One-Source Multi Application 등 주요 구현 범위를 정리해야 했습니다.

My Role

Role Scope

모바일 파트에서 Android/iOS 개발 범위, 구현 문서, 주요 기능의 협업 커뮤니케이션을 정리했습니다.

- SSO, One-Source Multi Application, SDK/API/WKWebView 연동 등 주요 모바일 기능 구현에 참여했습니다.

Key Contributions

- SSO Flow, 플랫폼별 구현 범위, scheme/환경 분리 방향을 정리했습니다.
- FCM, 외부 SDK, 소셜 로그인, API/WKWebView 연동 기능을 구현했습니다.

Technical Decisions

- One-Source Multi Application과 운영 전환 리스크를 고려해 scheme, 환경, SSO 연동 범위를 점검했습니다.
- Android/iOS 공통 요구사항과 플랫폼별 차이를 분리해 개발 문서와 구현 범위를 맞췄습니다.

Screenshots / Result

- 차세대 커머스 앱 전환 결과 화면입니다.

GS FRESH 차세대 커머스 앱 화면



Result / Impact

- SSO, 환경 분리, 외부 SDK/API 연동 등 모바일 구현 범위와 오픈 전 확인 기준을 정리했습니다.

Early Career Experience

초기 **ANDROID/SI** 프로젝트는 현재 **IOS** 시니어 포지셔닝을 흐리지 않도록 **COMPACT**하게 정리했습니다.

CafeUnion Android Application 개발

2016.08 ~ 2017.01

카페유니온은 중국 시장에 출시될 커피 브랜드 가맹형 O2O 프리오더 서비스 플랫폼 입니다. 국내의 스타벅스 커피 앱과 유사한 Application 입니다.

Android

Java

WeChat SDK

QQ SDK

Baidu SDK

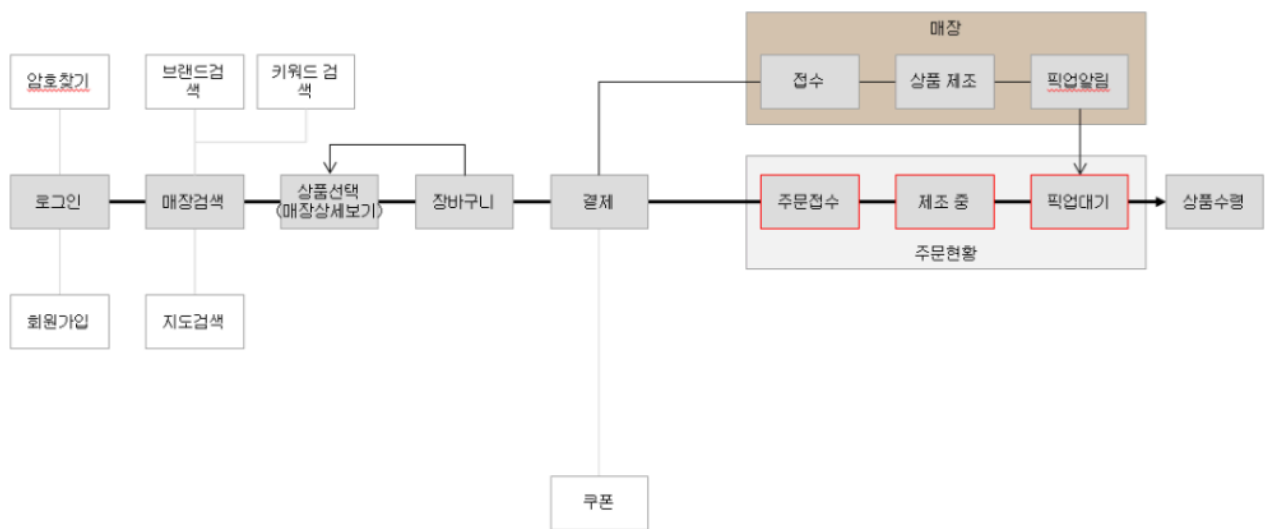
Alipay SDK

Picasso

Glide

gson

요구사항



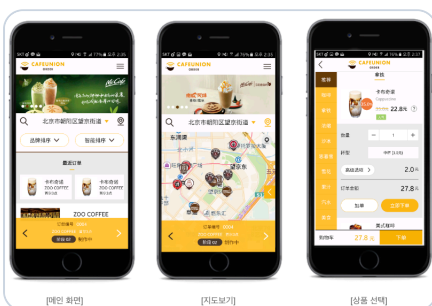
- 1. 현재 사용자의 위치를 기반으로 근접한 거리에 있는 매장 리스트 노출
- 2. 알리바바 지도 연동을 통한 위치 서비스 제공
- 3. 특정 매장 선택 시, 해당 매장에서 판매하고 있는 상품 리스트 노출
- 4. 사용자사 선택한 상품의 수량, 사이즈, 추가옵션 등 선택을 통해 해당 상품 장바구니 담기 및 결제 기능 구현

상세 진행 내용

- Android Application 개발
- API 연동
- 화면 UI 개발
 - 90% 이상 Native 화면으로 구성되어 있었으므로, 모든 페이지의 화면 UI 개발
 - N depth Expandable ListView 구현
 - Swipe Delete 기능 구현
 - Indexable ListView 구현
 - Infinite ViewPager 구현

- 중국 SNS 연동
 - 위챗, 웨이보, QQ 등 중국 소셜 로그인 및 회원가입 기능 개발
- 매장 찾기 및 네비게이션 기능 개발
 - 바이두 SDK 연동
 - 바이두 SDK를 활용한 네비게이션 기능 구현
- 바이두 SDK를 활용한 푸시 기능 개발
- WKWebView 연동
- 중국 결제 수단 연동
 - 위챗 페이, 알리페이 등 중국의 소셜 Pay 기능 연동을 위한 SDK 삽입 및 기능 개발

결과물



LuxeWater Android Application 개발

2015.11 ~ 2015.12

중국 마켓 워터 서플라이 플랫폼 럭스워터 Android Application을 개발했습니다.

Android

Java

Baidu SDK

QQ

Alipay SDK

Wechat SDK

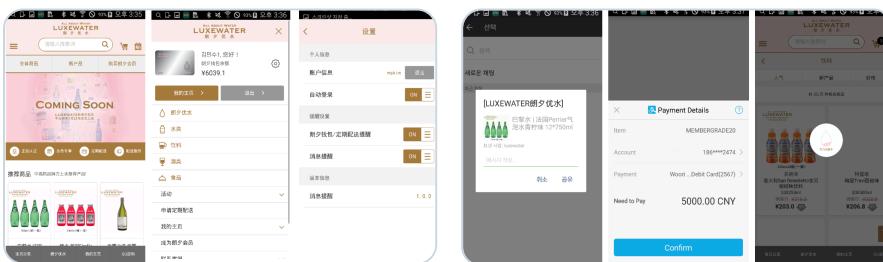
✓ 프로젝트 배경 설명

전체 화면 구성이 WebView로 구성 되어 있는 Hybrid Application 으로써, 중국 앱 마켓에 출시되었습니다. 중국이라는 국가적 특성에 맞춰 지도, 결제, 푸시, SNS 등의 서비스를 환경에 맞는 SDK로 삽입하여 기능을 구현했습니다.

상세 진행 내용

- Android Application 개발
- WebView 연동
 - WebView 연동을 위한 기본 세팅 개발
 - Javascript Interface 설계 및 연동
- API 연동
- 중국 SNS 연동
 - 위챗, 바이두, QQ 등 중국 소셜 로그인 및 회원가입 기능 개발
- 바이두 SDK를 활용한 푸시 기능 개발
- 중국 결제 수단 연동
 - 중국의 소셜 Pay 기능 연동을 위한 알리페이 SDK 삽입 및 기능 개발

결과물



올가홀푸드 Android Application 개발

2016.04 ~ 2016.05

기존 올가홀푸드 웹사이트를 모바일용 채널을 확보해 고객 유입을 유도하고자 구축한 올가홀푸드 Android Application 개발을 담당하였습니다.

Android

Java

FingerPush

ZXing

gson

universal image loader

viewpagerindicator

gms

crashlytics

✓ 프로젝트 배경 설명

기존 올가홀푸드 웹사이트는 모바일용 페이지 부재 등 모바일 환경에 대한 콘텐츠 접근성이 부족했습니다. 기존 웹사이트를 개선하고 모바일 환경을 제공함으로써, 세일즈를 강화하여 고객 유입을 유도할 수 있으며, 급변하는 모바일 커머스 시장에 대한 대응을 통해 안전한 먹거리라는 브랜드 아이덴티티를 전달하여 소비자와의 신뢰를 강화하고자 하였습니다.



상세 진행 내용

- Android Application 개발
- 화면 UI 개발
 - 전시 화면 개발
 - 사이드 메뉴 화면 개발
 - 제품 바코드 인식을 위한 스캔 기능 구현 및 화면 개발
 - 설정 화면 개발
- API 연동
- WebView 연동
 - WebView 연동을 위한 기본 세팅 개발
 - Javascript Interface 설계 및 연동
- FingerPush SDK를 활용한 푸시 기능 개발
- 제품 바코드 및 QR 코드 인식 기능을 위한 기능 개발
 - Zxing barcode scan library 연동

결과물



사내 인트라넷 Android Application 개발

2015.07 ~ 2015.09

사내 인프라넷 웹사이트를 기반으로 임직원들의 편의를 위해 제작된 인트라넷 Android Application 개발을 담당하였습니다.

Android

✓ 프로젝트 배경 설명

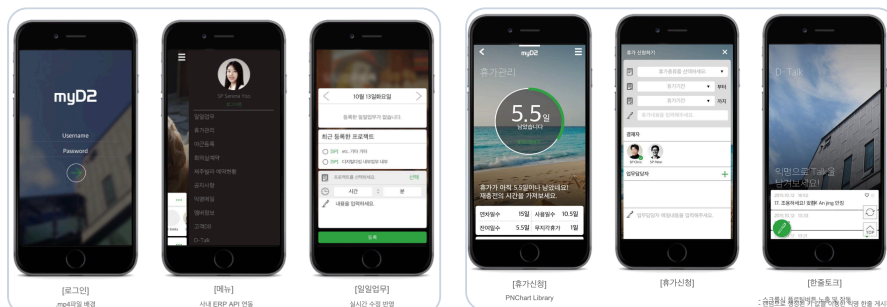
인트라넷 앱은 업무 커뮤니케이션 강화와 더불어 사내 커뮤니케이션을 활성화하여 정서적인 교류도 함께할 수 있는 플랫폼 구축을 목표로 했습니다. 더불어 사원들이 굳이 데스크탑을 사용하지 않아도 내부 정보나 업무를 효율적으로 진행할 수 있고자 했습니다.

일일 업무 등록, 휴가 결재, 야근 신청, 회의실 예약, 한줄 토크 등 다양한 인트라넷 기능을 모바일 네이티브 앱으로 제작하여 편의성을 극대화 했습니다.

상세 진행 내용

- Android Application 개발
- 전체 화면 UI 개발
- API 연동
- 푸시 기능 연동을 위한 GCM 연동

결과물



ONEstory iOS Application 운영 및 개발

2021.09 ~ 2023.12

원스토리는 웹툰, 웹소설, 만화, 일반 도서, 오디오북까지 다양한 콘텐츠를 즐길 수 있는 플랫폼 서비스입니다. iPad까지 호환되며, 위젯을 통해 최근 본 작품을 빠르게 이어볼 수 있는 기능부터 비행기모드/오프라인에서도 다운로드 한 작품을 즐길 수 있는 기능을 제공합니다.

iOS

Swift

SwiftUI

KeychainAccess

Kingfisher

CryptoSwift

KissXML

GoogleSignIn

NaverLoginSDK

JWTDecode

Lottie

Amplitude

ADpopcorn

정기 운영 업무

- 정기 배포 진행
- 앱 안정성 개선 및 버그 수정
- 신규 기능 개발 및 배포
- OS 업데이트 및 Deprecated Function 대응
- Crash Report 분석을 통한 앱 장애 요소 예방 및 대응
- 앱 개선 요청 사항 개발 대응

개인 담당 업무

- 내 서재, 구매 목록, 마이 페이지 feature 담당

 상세 설명

- 다운로드 한 도서 DB 관리 및 개발
- 3rd party 로그인 SDK 관리 및 개발
- 카카오톡 고객센터의 CS 화면 개발 및 운영
- 기존 웹뷰로 개발된 구매 목록화면을 Native UI로 화면 개발 및 전환

- Java Script Interface 설계 및 개발

- 위젯 개발 및 운영 담당

 상세 설명

- iPhone & iPad 위젯 개발 및 운영

- 콘텐츠 뷰어 추가 기능 개발

 상세 설명

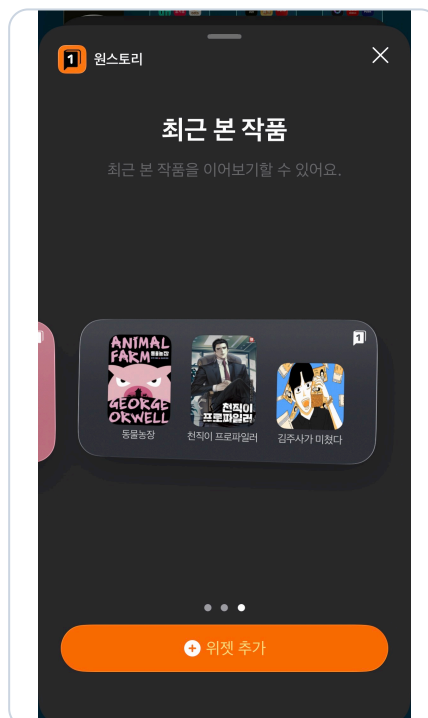
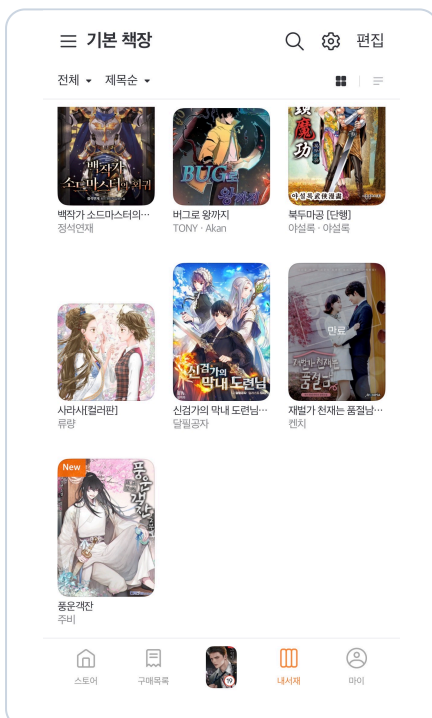
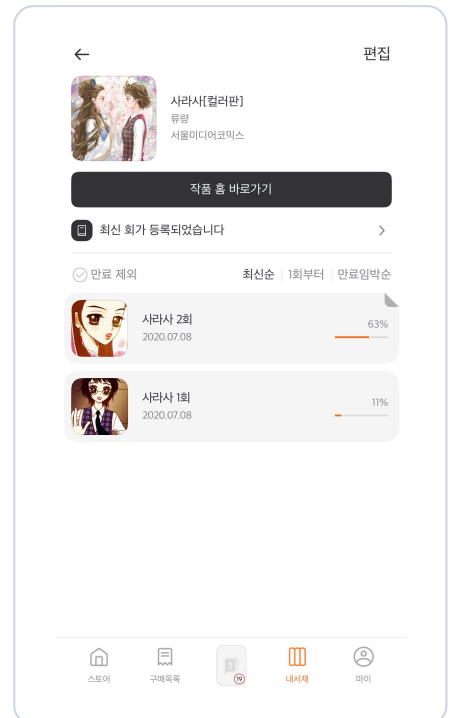
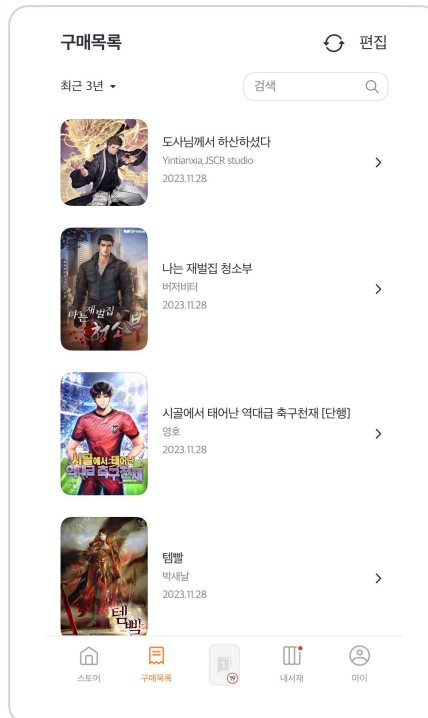
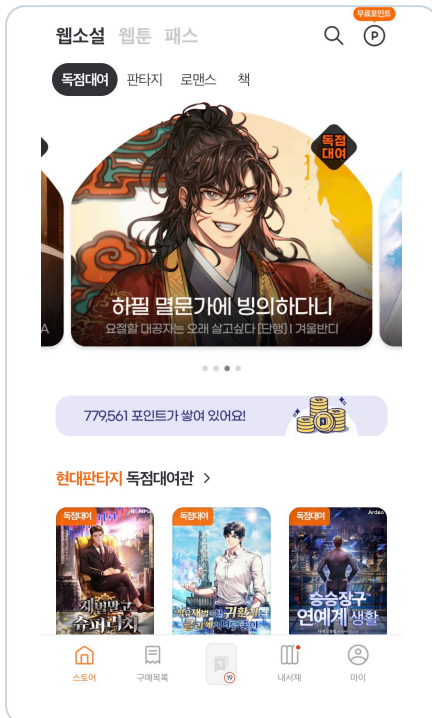
- Eye Tracking SDK를 활용한 웹툰 뷰어 자동 스크롤링 기능 구현

- 운영 업무 효율을 위한 업무

 상세 설명

- QA 및 개발 테스트를 위한 환경 세팅 조작 및 컨트롤 가능한 Application 개발 및 운영
- 외부 연동 규격 테스트를 위한 Tracer Application 개발 및 운영

 App Store

 다운로드 App Store 
 결과물


GS Fresh 통합 멤버십 앱 운영 및 개발

2018.08 ~ 2021.09

THE POP, GS수퍼마켓, GS25-나만의 냉장고, 우리동네 딜리버리 4가지 멤버십 어플리케이션을 운영 및 개발하는 업무를 담당했습니다.

iOS

Swift

RxSwift

RxCocoa

RxWebkit

RxViewController

ZXingObjc

Alamofire

Moya

ReachabilitySwift

KeychainAccess

CryptoSwift

Adjust

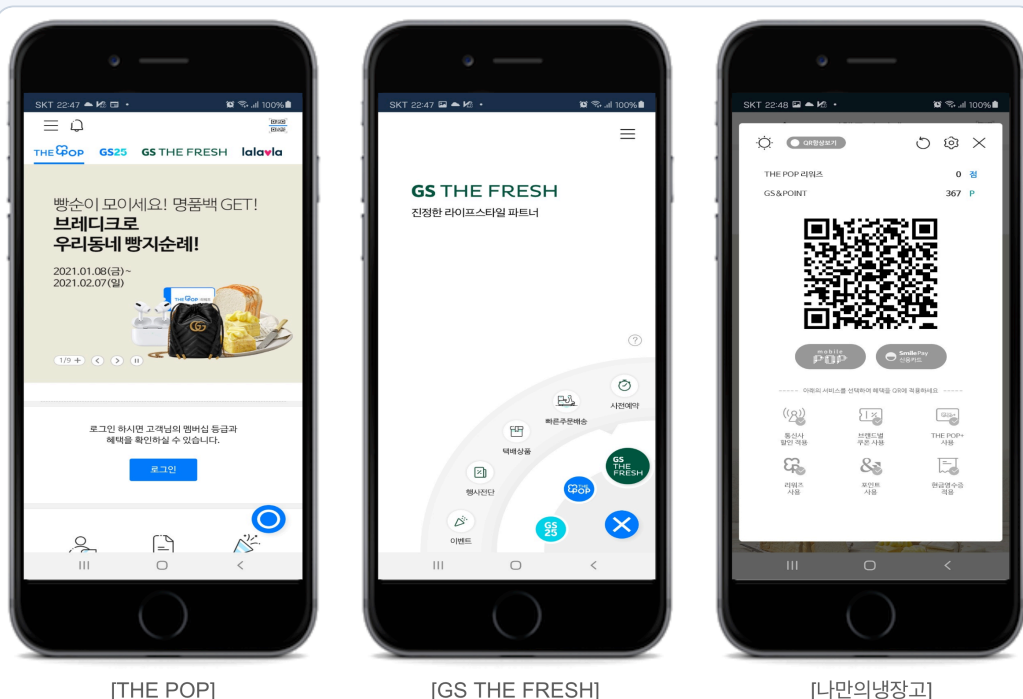
Firebase

THE POP, GS 수퍼마켓, 나만의냉장고는 GS리테일의 통합 멤버십 어플리케이션으로써, GS리테일 통합 커머스 앱과 같이 OneSource-Multi System을 이루고 있습니다. GS리테일 멤버십 이용, 각 채널의 쿠폰 사용, 상품 결제 등을 할 수 있습니다.

정기 운영 업무

- 월 2회 정기 배포 진행, 긴급 배포 상시
- VOC 대응 및 오류 대응
- 신규 OS Update 대응 및 Deprecated Function 대응
- Crash Report 분석을 통한 앱 장애 요소 예방 및 대응
- 현업의 앱 개선 요청 사항 개발 대응
- 운영 시스템 관련 프로젝트에 대한 운영팀으로서의 개발 Inspection 리딩
- 성능 및 안정성 개선
- 신규 기능 개발 및 배포

결과물



AI-Assisted Project Workflow

AI를 코드 생성 도구로만 사용하지 않고, 컨텍스트 설계·단계별 구현·검증 루프를 갖춘 개발 워크플로우로 활용합니다.

Context Engineering

요구사항, API 계약, 디자인 기준, 아키텍처 규칙을 먼저 정리해 AI가 같은 기준을 참조하도록 만듭니다.

Token-efficient Prompting

큰 기능은 책임 단위로 나누고, 현재 단계의 변경 목표와 금지 조건만 전달해 불필요한 컨텍스트와 diff를 줄입니다.

Guardrailed Implementation

수정 대상 파일, 금지 파일, 보안 기준, 레이어 경계를 지정해 AI 결과가 전체 구조 개편으로 번지지 않도록 제어합니다.

Developer-owned Verification

AI 결과는 초안으로 보고, Xcode build/test, git diff/status, 실제 앱 실행, 수동 QA로 직접 검증합니다.

Side Project

학습지 지우개 - Note Cleaner

전체 2025.11 ~ 2026.03 · 개발 2026.01 ~ 2026.02

영어 학습지 이미지를 VisionKit으로 입력받고, OpenAI Responses API용 프롬프트와 응답 DTO를 직접 설계해 triage와 정리 결과로 연결한 App Store 출시 개인 사이드 프로젝트입니다.

iOS Swift SwiftUI VisionKit OpenAI API Prompt Design Structured Response
Clean Architecture MVVM RxSwift Firebase Remote Config Fastlane

Project Context

- 전체 2025.11 ~ 2026.03, iOS 개발 집중 기간은 2026.01 ~ 2026.02입니다.
- Android 1명, iOS 1명, 기획 2명, 디자이너 1명 구성에서 iOS 앱 설계, 구현, App Store 배포를 담당했습니다.

Problem

- 모든 이미지를 바로 AI 처리로 넘기면 비용, 정확도, 실패 케이스 관리가 어려워질 수 있었습니다.
- 영어 학습지 여부, 이미지 품질, 후속 정리 필요 여부를 먼저 판별하는 triage 흐름이 필요했습니다.

My Role

- 문제 정의, iOS 앱 플로우, VisionKit 기반 문서 이미지 입력 흐름을 설계·구현했습니다.
- 필기 제거용 OpenAI 프롬프트와 응답 DTO를 설계하고, ACCEPT / REJECT 기반 triage와 reason_code 검증 흐름을 구성했습니다.
- 토큰 사용량을 줄이기 위해 출력 형식과 응답 스키마를 정리하고, SwiftUI 상태 처리와 App Store 배포까지 담당했습니다.

Architecture

- Presentation, Domain, Data 레이어로 나누고 SwiftUI View, @Observable @MainActor ViewModel, UseCase, Repository Protocol을 분리했습니다.
- Data 레이어에는 OpenAI API 클라이언트, Repository 구현체, Firebase Remote Config, FileManager/UserDefaults 저장 흐름을 배치했습니다.

Key Implementation

- VisionKit 스캔 후 네트워크 상태를 확인하고, ExtractWorksheetTextUseCase를 통해 OCR/정리 요청을 실행했습니다.
- OpenAI Responses API는 URLSession 기반 커스텀 클라이언트로 연동하고, 요청 전 이미지 리사이즈와 JPEG 압축을 적용했습니다.
- status, reason_code, confidence, extracted_markdown, truncated, truncation_reason 중심의 DTO와 timeout/HTTP/parse/model refusal 오류 흐름을 분리했습니다.
- 프롬프트는 처리 대상 판단, 필기 제거 기준, 출력 형식을 나누고 불필요한 설명 출력을 줄이는 방향으로 정리했습니다.

AI Integration

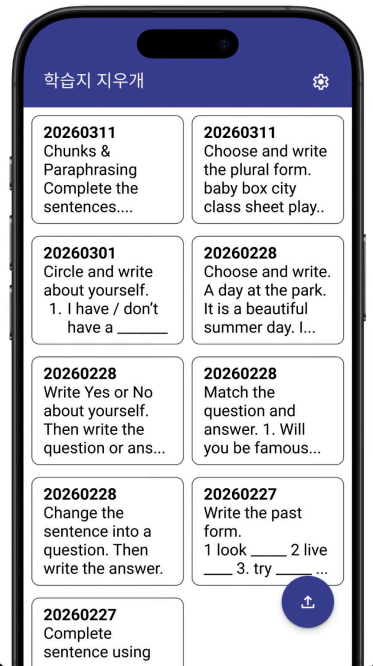
- ACCEPT / REJECT와 reason_code 조합을 앱에서 다시 검증해 AI 응답을 그대로 신뢰하지 않도록 설계했습니다.

- truncated와 truncation_reason을 분리해 출력 제한 상황을 다루고, 잘못된 입력은 설명 가능한 실패 상태로 연결했습니다.

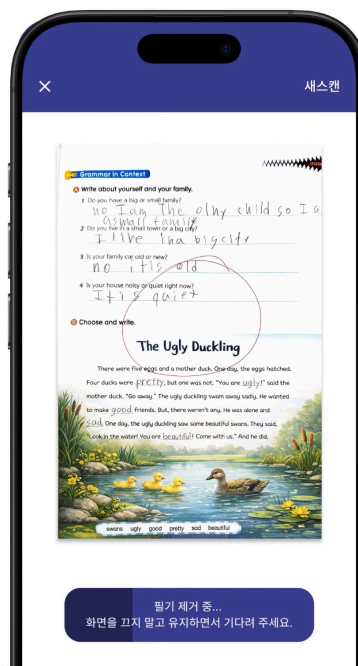
Screenshots / Result

APP STORE RELEASE SCREENS

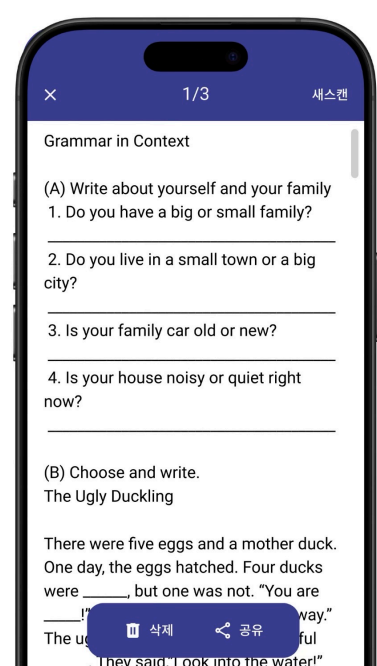
AI 학습지 지우개로
학습지를 필기 전
상태로 돌려세요



학습지 필기를
빠르고 정확하게
지울 수 있어요



원본 학습지를
공유하거나 ★❤️
다시 풀 수 있어요 ✓



Result

- AI API를 iOS 앱 플로우 안에 방어적으로 연결하고 App Store 출시까지 진행했습니다.
- 기획, 디자인, Android 개발자와 협업하며 iOS 구조와 출시 흐름을 정리했습니다.

App Store

App Store에서 보기

Mindly-iOS

2026.05 ~ 진행 중 · iOS 1 / Android 1 / Planner 1

저장한 링크와 메모를 AI 태그와 로컬 검색 인덱스로 정리해, 나중에 키워드·태그·자연어로 다시 찾을 수 있게 만든 iOS 키워드 메모 앱입니다.

iOS

Swift

SwiftUI

GRDB

Apple Intelligence

Firebase Functions

OpenGraph

Share Extension

Codex

AGENTS.md

Skills

Overview

- 웹서핑이나 일상 중 발견한 링크와 짧은 메모를 저장하고, AI가 자동으로 태그와 검색용 메타데이터를 생성해주는 iOS 앱입니다.
- 노트는 GRDB 기반 로컬 DB에 먼저 저장되며, AI 태그 생성은 비동기로 실행되어 사용자의 저장 흐름을 막지 않도록 설계했습니다.
- 기본 검색은 로컬 키워드/태그 검색으로 동작하고, AI 검색은 자연어를 검색 조건으로 변환한 뒤 로컬 DB에서 결과를 조회합니다.
- Share Extension을 통해 Safari나 외부 앱에서 링크/텍스트를 바로 Mindly 저장 흐름으로 연결할 수 있도록 구현했습니다.

Problem

- 좋은 링크나 짧은 아이디어를 저장해도 시간이 지나면 어떤 키워드로 다시 찾아야 할지 기억하기 어렵습니다.
- 저장 시점의 링크 정보, 메모, 태그, 검색 인덱스를 함께 정리해 재탐색 가능한 구조가 필요했습니다.

My Role

- iOS 개발자로 제품 기획 정리, iOS 앱 구조 설계, SwiftUI 화면 구현, GRDB 로컬 저장소, AI 태그/검색 흐름, Share Extension, 실기기 QA를 담당했습니다.
- 총 3명 규모(iOS 1명, Android 1명, 기획 1명)에서 iOS 저장-태그 생성-검색-상세 관리 흐름을 구현했습니다.
- Apple Intelligence와 Firebase Functions를 활용해 AI 태그 생성 및 AI 검색 fallback 구조를 설계했습니다.

Codex Development Workflow

Context Design

AGENTS.md에는 MVVM + UseCase + Repository, GRDB/FTS5 local search, AI provider, 민감 정보 로그 금지처럼 반복 규칙만 두고, PRD/Firebase/design 상세는 docs로 분리했습니다.

Skills

feature-plan, ios-implementation, firebase-functions, grdb-search, swiftui-screen-scaffold, security-review skill로 계획·구현·Firebase·검색·화면·보안 검토를 분리했습니다.

Prompt & Token Strategy

"계획만 보고", "문서만 수정", "이 파일 외 수정 금지", "fallback 추가 금지"처럼 범위를 좁혔고, 필요한 문서와 이전 QA 결과만 제공해 컨텍스트를 줄였습니다.

Guardrails

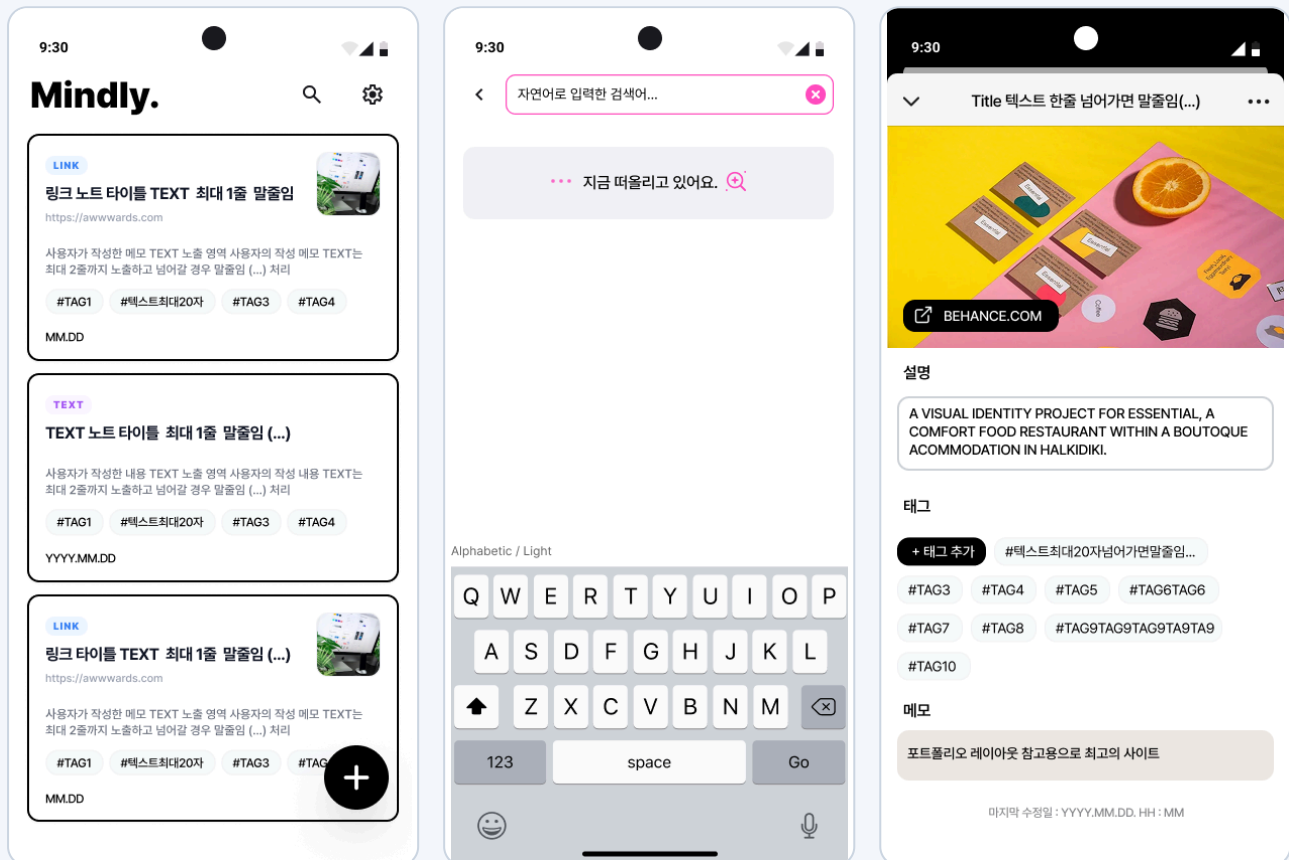
View의 GRDB/Firebase/FoundationModels 직접 호출을 막고, OpenAI API Key는 iOS 앱에 넣지 않도록 Firebase Functions를 fallback 경계로 고정했습니다. default search는 local-only로 유지했습니다.

Verification

- Codex 결과는 git diff/status, Xcode build/test, Preview, iPhone 15 실기기 QA, Firebase Console 로그, 디자인 시안 비교로 검증했습니다.
- UI 이슈는 스크린샷 기반으로 확인했고, 잘못된 변경은 즉시 원복했습니다.

Screenshots / Result

MINDLY IOS SCREENS



노트 리스트, 자연어 기반 AI 검색, 링크 상세 화면을 통해 저장-탐색-상세 관리 흐름을 확인할 수 있습니다.

Result / Status

- 문서 체계, DesignSystem, SwiftUI 화면, GRDB 저장/검색 구조, AI 태그 생성/검색 흐름, Share Extension까지 iOS 앱 구조를 단계적으로 구성했습니다.
- AI 도구를 활용하되 최종 설계 판단과 품질 검증은 직접 수행하는 개발 워크플로우 사례로 정리할 수 있습니다.

FocusBoard

Side Project · SwiftUI macOS App · App Store Released

출석, 집중 타이머, Todo, 통계, 메뉴바, 위젯을 포함한 macOS 생산성 앱을 Codex 기반 작업 규칙과 단계별 검증 흐름으로 설계·구현하고 App Store에 출시한 프로젝트입니다.

Swift SwiftUI macOS MenuBarExtra Widget App Group JSON Persistence
Firebase Remote Config String Catalog Codex AGENTS.md Codex Agents Skills

Overview

- 출석 체크, 집중 타이머, Todo, 반복 일정, 리마인더, 통계, 메뉴바, 위젯, 원형 시간표를 제공하는 macOS 생산성 앱 프로젝트입니다.
- Codex를 단순 코드 생성 도구가 아니라 기능 기획, 단계별 구현, 리뷰, 버그 수정, 테스트 보강, 배포 준비 문서 작성까지 이어지는 개발 보조 시스템으로 활용했습니다.
- 개인 사이드 프로젝트로 App Store에 출시했습니다.

My Role

- 기능별 kickoff 문서를 통해 goal, policy, file boundaries, risks, out-of-scope를 먼저 정리했습니다.
- Attendance, Focus Timer, Todo, Statistics, MenuBarExtra, Widget, Schedule 기능을 작은 단계로 나누어 구현했습니다.

Codex Development Workflow

Context Design

AGENTS.md에는 macOS-only, local-first, JSON persistence, 레이어 경계처럼 장기 규칙만 두고, 긴 PRD·디자인 수치·Firebase JSON·임시 의사결정은 단계별 프롬프트와 kickoff 문서로 분리했습니다.

Codex Agents

implementation_reviewer, apple_surface_specialist, persistence_and_stats agent를 구성해 구현 리뷰, quick surface 경계, 저장소·통계 리스크를 분리해 검토할 수 있게 했습니다.

Skills

feature-kickoff, implement-quick-surface, ship-check skill로 기능 시작, MenuBarExtra/Widget 같은 quick surface 구현, 릴리즈 전 점검을 반복 가능한 흐름으로 분리했습니다.

Prompt & Token Strategy

"kickoff부터", "이번 단계는 Domain만", "UI 제외", "이 파일 외 수정 금지", "다음 단계 선행 구현 금지"처럼 단계와 파일 범위를 좁혀 컨텍스트와 diff를 작게 유지했습니다.

Guardrails

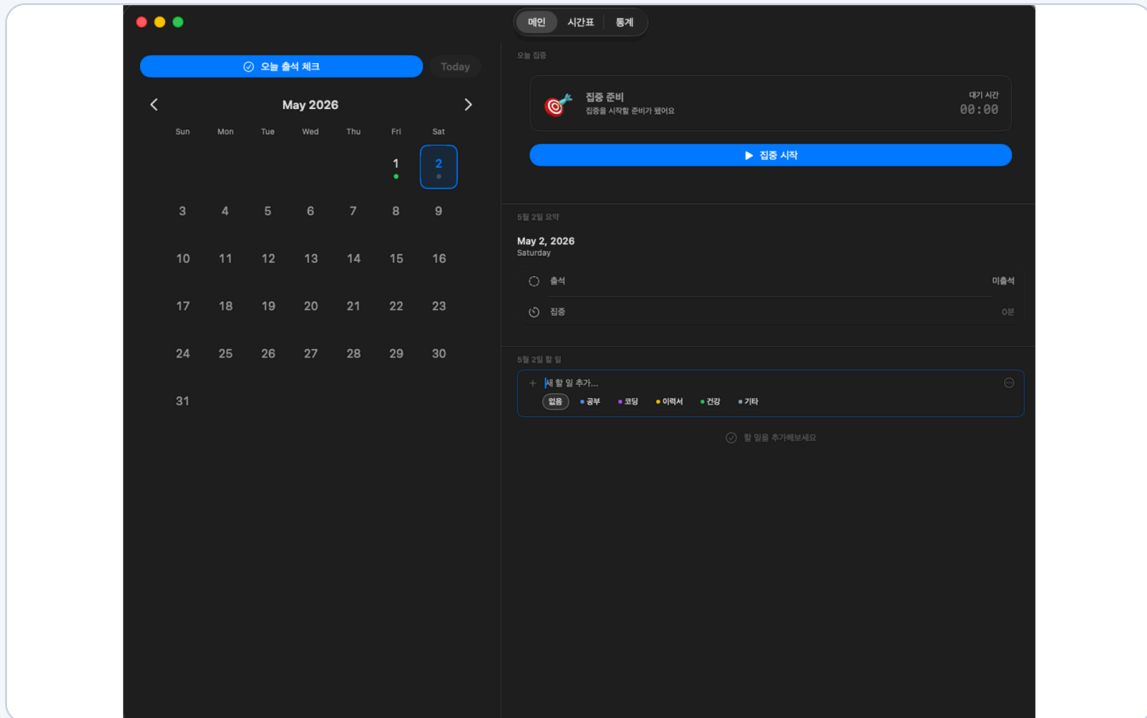
Domain import 제한, SwiftData 도입 금지, Firebase 사용 범위 제한, 원치 않는 리팩토링 금지, quick surface business logic 금지를 반복해 기능 추가가 전체 구조 개편으로 번지지 않게 했습니다.

Verification

- Codex 결과는 git diff/status, Xcode build/test, 실제 macOS 앱 실행, SwiftUI Preview, Firebase Console 로그, 수동 QA 체크리스트로 검증했습니다.
- Preview, String Catalog, Firebase, Widget 이슈는 핵심 에러와 증상만 다시 제공해 최소 수정하도록 유도했습니다.

Screenshots / Result

FOCUSBOARD APP STORE SCREENSHOT



출석 캘린더, 집중 타이머, 오늘 요약, Todo 입력 흐름이 한 화면 안에서 연결되는 macOS 앱 구조를 보여주는 스크린샷입니다.

Result / Status

- macOS 앱 구조, MenuBarExtra, Today Summary Widget, JSON 공유 저장소, Firebase Remote Config 버전 체크, 다국어 대응, App Store 제출 자료를 정리하고 출시까지 진행했습니다.
- AI를 기획 보조와 구현 보조에 활용하되, 단계 분리와 직접 검증을 통해 변경 리스크를 관리한 사례입니다.

App Store

App Store에서 보기

About Me

커머스, 콘텐츠, 플랫폼 도메인에서 모바일 앱을 개발·운영하며 안정성, 유지보수성, 협업 비용을 함께 봅니다. Android 개발자로 시작해 iOS로 전향하는 과정에서 두 플랫폼의 화면 구조, 배포/운영 방식, SDK/API 연동 차이를 직접 학습했고, 이 경험은 이후 iOS 앱 구조와 운영 흐름을 이해하는 기반이 되었습니다.

디자이너, 기획자, 서버/웹/모바일 개발자와 요구사항을 맞추는 협업을 꾸준히 해왔으며, Android/iOS 양쪽의 구현 흐름을 이해한 경험을 바탕으로 모바일 파트의 사용자 시나리오와 플랫폼별 기술 스펙을 맞추는 조율 역할을 수행했습니다.

Operation First

운영 이슈는 재현 조건, 영향 범위, 배포 리스크부터 분리합니다. Crashlytics, API 로깅, VOC를 원인 추적 단서로 보고 사용자 영향이 큰 문제부터 개선합니다.

Incremental Modernization

레거시는 유지되어 온 이유와 의존성을 먼저 확인합니다. 전체 교체보다 영향 범위를 나누고 SwiftUI, MVVM 등으로 옮길 수 있는 안전한 변경 단위를 찾습니다.

Context-aware Technology Choice

UIKit, SwiftUI, RxSwift, Combine은 상황에 맞는 선택지로 봅니다. 기존 코드, 팀 운영 방식, 테스트 가능성, 유지보수 비용을 기준으로 적용 범위를 정합니다.

Platform Collaboration

반복되는 구현과 논의는 디자인 시스템, 공통 컴포넌트, 사용 기준으로 구조화하는 방식을 선호합니다. 협업 과정에서는 구현 범위와 예외 케이스를 명확히 맞춥니다.

AI-assisted, Developer-owned

AI 도구는 레거시 분석, 설계 검토, 테스트 초안 작성의 속도를 높이는 데 활용합니다. 최종 판단과 품질 책임은 기존 코드와 운영 리스크를 이해한 개발자가 가져가야 한다고 봅니다.

Tech Stack

Core iOS

Swift, Objective-C, UIKit, SwiftUI, Combine, RxSwift

Architecture

Clean Architecture, MVVM, MVI, Reactor-style Flow, DI, Coordinator

Networking / Persistence

URLSession, Moya, Alamofire, RxMoya, RxAlamofire, Keychain, Core Data, SwiftData

Operation / Quality

Firebase, Crashlytics, API Logging, Fastlane, XCTest, Swift Testing

Design System

Figma, Design Token, Style Dictionary, Figma Code Connect

AI Workflow

Claude Code, Cursor, Codex, Copilot, ChatGPT, Figma Make, Pencil

Contact

Email amy.lee.d2@gmail.com

Phone +82 10-6345-4342